

COS 214 Practical 3: EventFlow

Task 4: Event Rules and Dynamic Behaviour (25 marks)

Team members: Chitambala Sikazwe (u25173384),
Gary Mull (u25071247), Louwrens Johansen (u25607414)

This task builds directly on Task 3's Observer classes: EventNotice (a struct with type, senderName, message and severityLevel), the Observer and Subject base classes, and CascadingZone, the class that combines a Composite group with the Subject and Observer roles so it can receive a notice from above and relay it to its own registered observers below. Nothing in Task 3 was changed. The five concrete leaves from Task 2 were extended to also implement Observer, and CascadingZone gained one new method, transferChild(), for the runtime reorganisation this task asks for.

4.1 Five event rules

Each of the five leaf classes now implements update(const EventNotice&) and reacts differently to the same notice, through polymorphism rather than a type check in a controller. The clearest example is a WEATHER_ALERT:

- MainStage: pauses the performance (close()). An outdoor stage cannot safely keep running.
- SecurityGate: has no rule for WEATHER_ALERT specifically, only for CAPACITY_ALERT and EVACUATE, so it logs that it has no reaction and stays as it was. This is itself a deliberate difference: not every unit treats every notice as relevant.
- FoodVendor: suspends kitchen service (close()).
- FirstAidTent (MedicalTeam): stays active. If the notice's severityLevel is 4 or higher it also switches to backup power, but it never closes.
- ShuttleStop: reroutes to an emergency path and keeps running, rather than stopping.

This is visible directly in the demonstration output (Task4Testing.cpp, testTask4()): a single call to rootFestival->issueNotice(WEATHER_ALERT, ...) cascades through Main Stage Zone and River Side Zone and results in the stage stopping, the medical tent switching to backup power, the vendor halting service and the shuttle rerouting, all from the one notice. Because Food Court Precinct is itself a CascadingZone attached to River Side Zone, the same call also shows the notice reaching a leaf (the Taco Stand) three runtime levels below the root, which is the same cascading behaviour Task 3 requires and confirms the two tasks are working from the same notification chain.

4.2 Runtime reorganisation

CascadingZone::transferChild(EventComponent* child, CascadingZone* newParent) moves the Taco Stand vendor from the Food Court Precinct to the Main Stage Zone while the program is running. It updates two things that must both change together:

- Composite ownership: remove(child), inherited from EventComposite, takes the pointer out of the old zone's children vector without deleting the object, then newParent->add(child) gives ownership to the new zone.

- Observer registration: if the child is also an Observer, detach(childObserver), inherited from Subject, removes it from this zone's observer list (safely doing nothing if it was not registered there), and newParent->attach(childObserver) registers it with the new zone.

The demonstration shows this is not just cosmetic. After the transfer, the Taco Stand appears in Main Stage Zone's status report instead of Food Court Precinct's, and when a later OPEN notice is sent, the Taco Stand reopens as part of Main Stage Zone's cascade, not Food Court Precinct's, which confirms the observer registration actually moved and not only the ownership pointer.

4.3 A condition-based decision

SecurityGate::update() contains the condition this task asks for, ready to become an alt fragment with guards in Task 5's SD3:

```
if (notice.severityLevel >= closeThreshold) { close(); } else { activateVipLane(); }
```

A CAPACITY_ALERT with a severityLevel below the gate's own closeThreshold (4) only restricts admission to a single VIP lane (see 4.4 below). A severityLevel at or above the threshold closes the gate completely. The demonstration sends both cases directly at the same gate object, first with severityLevel 2 and then with severityLevel 4, to show both branches actually run.

4.4 Three original features

1. SecurityGate VIP fast-track lane. Instead of a binary open/closed gate, a moderate CAPACITY_ALERT restricts the gate to a single fast-track lane (activateVipLane()) rather than shutting it completely, which is closer to how a real festival gate would behave under pressure. The logic lives entirely inside SecurityGate: it only adds one boolean flag, one threshold constant and one private method to a class that already owned this decision, so it does not turn SecurityGate or anything else into a god object.

2. MedicalTeam backup power. A severe WEATHER_ALERT (severityLevel 4 or higher) makes FirstAidTent switch to backup power (activateBackupPower()) instead of doing nothing or closing, reflecting that a safety-critical unit needs a stronger response than an ordinary one. Again, the flag and the method are private to FirstAidTent and nothing outside that class needs to know backup power exists.

3. ShuttleStop dynamic rerouting. Rather than simply stopping like the other units, ShuttleStop reacts to WEATHER_ALERT and EVACUATE by rerouting itself onto an emergency path (rerouteToEmergencyPath()) and continuing to run, since transport is one of the few things attendees need more of during an incident, not less. This stays a single method on ShuttleStop and does not require EventControl, EventComposite, CascadingZone or any other class to know about routes at all.

All three features are contained inside the one leaf class they belong to. Neither EventComposite nor CascadingZone ever inspects a concrete leaf type or branches on which feature a unit supports; they only ever call open(), close(), reportStatus(), getCapacity() or update() through the EventComponent/Observer interfaces, so adding these features did not require touching any of Task 2 or Task 3's own code.

A note on notice types

Task 3's NoticeType enum covers OPEN, CLOSE, SCHEDULE_CHANGE, CAPACITY_ALERT, WEATHER_ALERT and EVACUATE, six types in total, which satisfies the practical's minimum of six without PAUSE or RESUME. Task 4's event rules use OPEN as the signal that restores normal operation (reopening a stage, gate, vendor or shuttle, and switching the medical tent off backup power), which fits naturally alongside CLOSE and EVACUATE without needing any change to Task 3's enum.